COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/24/26	**DATE SUBMITTED:**	07/24/26
NAME:	MATT JOHN ACALA	**INSTRUCTOR:**	Engr. Jimlord

ACTIVITY NO. 2 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

Screenshot

```
%%writefile main.cpp
#include<iostream>
#include<utility>

class Node{
public:
char data;
Node *next;
};


void ListTraversal(Node* head) {
    if (head == nullptr) {
        std::cout << "List is empty." << std::endl;
        return;
    }
    while (head != nullptr) {
        std::cout << head->data << " ";
        head = head->next;
    }
    std::cout << std::endl;
}


int main(){
//step 1
Node *head = NULL;
Node *second = NULL;
Node *third = NULL;
Node *fourth = NULL;
Node *fifth = NULL;
Node *last = NULL;
//step 2
head = new Node;
second = new Node;
third = new Node;
fourth = new Node;
fifth = new Node;
last = new Node;
//step 3
head->data = 'C';
head->next = second;
second->data = 'P';
second->next = third;
third->data = 'E';
third->next = fourth;
fourth->data = '0';
fourth->next = fifth;
fifth->data = '1';
fifth->next = last;
//step 4
last->data = '0';
last->next = nullptr;


ListTraversal(head);

return 0;
}
```

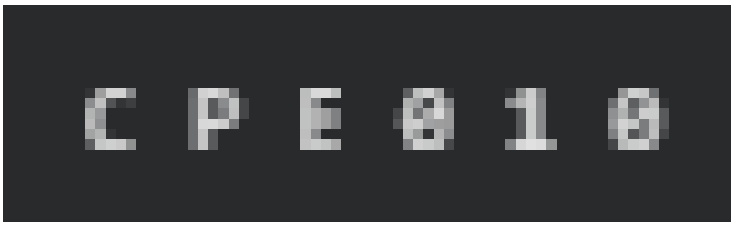
	
Discussion	Base on my understanding what i understand you just created a class to create a node then since its just single linked list it has no prev only data and next. And then i also created a void function to print all the data within the list of nodes using while loop. In the main function where i assigned the values per node but first i pointed it to null then reassigned it. After all of that i used the traverse function to print all the data of the list of nodes.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	`dllListTraversal`
Insertion at head	`dllInsertHead`
Insertion at any part of the list	`dllGeneralInsert`
Insertion at the end	`dllInsertTail`
Delete Node	`dllDeleteNode`

Table 3-2. Code for the List Operations

a. Source Code/Console

```
//Creating new node data
template <typename T>
DoubleList<T>* CreateNewNode(T NewData){
    DoubleList<T> *NewNode = new DoubleList<T>;

    NewNode->data = NewData;
    NewNode->next = nullptr;
    NewNode->prev = nullptr;

    return NewNode;
}
```

b. Source Code/Console

```
template <typename T>
void dllListTraversal(DoubleList<T>* head) {

    DoubleList<T> *tail;

    if (head == nullptr) {
        std::cout << "List is empty." << std::endl;
        return;
    }
    while (head != nullptr) {
        std::cout << head->data << " ";
        tail = head;
        head = head->next;
    }
    std::cout << std::endl;
    std::cout << "Backward: ";
    while(tail != nullptr){
        std::cout<< tail->data<<" ";
        tail = tail->prev;
    }

}
```

```
Try traversal: 1 2 3 4
Backward: 4 3 2 1
```

c. Source Code/Console

```
//inserting at head
template <typename T>
void dllInsertHead(T NewData, DoubleList<T>*& head){
    DoubleList<T> *NewNode = CreateNewNode(NewData);

    NewNode->next = head;
    head->prev = NewNode;
    head = NewNode;
}
```

```
Trying Insert at tail: 0 1 2 3 4 5
Backward: 5 4 3 2 1 0
```

d. Source Code/Console

```
//inserting at tail
template <typename T>
void dllInsertTail(T NewData, DoubleList<T>*& head){
    DoubleList<T>* NewNode = CreateNewNode(NewData);

    if (head == nullptr) {
        head = NewNode;
        return;
    }

    DoubleList<T>* current = head;
    while (current->next != nullptr) {
        current = current->next;
    }

    current->next = NewNode;
    NewNode->prev = current;
}
```

```
Trying Insert at tail: 0 1 2 3 4 5
Backward: 5 4 3 2 1 0
```

e. Source Code/Console

```
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous value cannot be null!" << std::endl;
        return;
    }
    DoubleList<T>* newNode = new DoubleList<T>();
    newNode->data = newData;
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    // Fix the next node's prev pointerr
    if (prevNode->next != nullptr) {
        prevNode->next->prev = newNode;
    }

    prevNode->next = newNode;
}
```

```
Trying Insert at 3rd node: 0 1 2 A 3 4 5
Backward: 5 4 3 A 2 1 0
```

f. Source Code/Console

```
//delete node
template <typename T>
void dllDeleteNode(T data, DoubleList<T>*& head) {
    if (head == nullptr) return;

    DoubleList<T>* temp = head;

    // search for the node
    while (temp != nullptr && temp->data != data) {
        temp = temp->next;
    }

    // Node wasn't found in the list
    if (temp == nullptr) return;

    // deleting the head
    if (temp == head) {
        head = temp->next;
    }

    // deleting a middle or tail node
    if (temp->prev != nullptr) {
        temp->prev->next = temp->next;
    }

    // deleting the tail node
    if (temp->next != nullptr) {
        temp->next->prev = temp->prev;
    }

    // deleting the node
    delete temp;
}
```

```
PS C:\Programming\C++\SCHOOL\CPE 010> cd
Try traversal: 1 2 3 4
Backward: 4 3 2 1
Try inserting at head: 0 1 2 3 4
Backward: 4 3 2 1 0
Trying Insert at tail: 0 1 2 3 4 5
Backward: 5 4 3 2 1 0
Trying Insert at 3rd node: 0 1 2 A 3 4 5
Backward: 5 4 3 A 2 1 0
Trying deleting 'A': 0 1 2 3 4 5
Backward: 5 4 3 2 1 0
```

Table 3-3. Code and Analysis for Double Linked Lists

Screenshots	Analysis

`//creating list for my name`
`DoubleList<char>* head = CreateNewNode('1');`
`DoubleList<char>* second = CreateNewNode('2');`
`DoubleList<char>* third = CreateNewNode('3');`
`DoubleList<char>* fourth = CreateNewNode('4');`
	This is where i created the list of my nodes and also using the function CreateNewNode to assign it values

`std::cout<<"Try traversal: ";` `dllListTraversal(head);`	This is where i printed all of the data of my nodes and also printing it backwards since its double linked list it has a prev

`std::cout<<"Try inserting at head: ";`
`dllInsertHead('0', head);`
`dllListTraversal(head);`
	This is where i inserted at head

`std::cout<<"Trying Insert at tail: ";`
`dllInsertTail('5', head);`
`dllListTraversal(head);`
	Opposite of the inserting at the head this is where i inserted at the tail or end of the node where the next is pointing at nullptr

`std::cout<<"Trying Insert at 3rd node: ";`
`dllGeneralInsert('A', head->next->next);`
`dllListTraversal(head);`
	This is where you will insert a node at a specific location other that head or tail since you can use the other function for that

`std::cout<<"Trying deleting 'A': ";`
`dllDeleteNode('A', head);`
`dllListTraversal(head);`
	This is where i delete a specific node it checks the data first then search it to the entire list of node then delete that node.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++
Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.
These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a

circular linked list.
We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- 24
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).
- Here are the steps to solve the problem:
- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.
- The driver function should allow for common operations on a playlist such as: next, previous, play all songs,
- insert and remove.

```
#ifndef CIRCULAR_DOUBLY_LL_H
#define CIRCULAR_DOUBLY_LL_H

template <typename T>
class DoubleList {
public:
    T data;
    DoubleList<T>* prev;
    DoubleList<T>* next;
};

// Creating new node data
template <typename T>
DoubleList<T>* CreateNewNode(T NewData) {
    DoubleList<T>* NewNode = new DoubleList<T>;
    NewNode->data = NewData;
    NewNode->next = nullptr;
    NewNode->prev = nullptr;
    return NewNode;
}
```

```cpp
// Circular Traversal (Forward & Backward)
template <typename T>
void dllListTraversal(DoubleList<T>* head) {
    if (head == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }

    std::cout << "\n========== PLAYLIST ==========\n";

    // Forward
    std::cout << "\nForward:\n";
    DoubleList<T>* current = head;
    int count = 1;

    do {
        std::cout << "[" << count++ << "] " << current->data << '\n';
        current = current->next;
    } while (current != head);

    std::cout << "\n=======Loops back to the first song=======\n";

    // Backward
    std::cout << "\nBackward:\n";
    current = head->prev;
    count--;

    do {
        std::cout << "[" << count-- << "] " << current->data << '\n';
        current = current->prev;
    } while (current != head->prev);

    std::cout << "\n=======Loops back to the last song=======\n";
}
```

```cpp
// Inserting at head
template <typename T>
void dllInsertHead(T NewData, DoubleList<T>*& head) {
    DoubleList<T>* NewNode = CreateNewNode(NewData);

    if (head == nullptr) {
        head = NewNode;
        head->next = head;
        head->prev = head;
        return;
    }

    DoubleList<T>* tail = head->prev;

    NewNode->next = head;
    NewNode->prev = tail;
    head->prev = NewNode;
    tail->next = NewNode;
    head = NewNode;
}
```

```cpp
// Inserting at tail
template <typename T>
void dllInsertTail(T NewData, DoubleList<T>*& head) {
    DoubleList<T>* NewNode = CreateNewNode(NewData);

    if (head == nullptr) {
        head = NewNode;
        head->next = head;
        head->prev = head;
        return;
    }

    DoubleList<T>* tail = head->prev;

    tail->next = NewNode;
    NewNode->prev = tail;
    NewNode->next = head;
    head->prev = NewNode;
}
```

```cpp
// General Insert after a given node
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null!" << std::endl;
        return;
    }

    DoubleList<T>* newNode = CreateNewNode(newData);
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    prevNode->next->prev = newNode;
    prevNode->next = newNode;
}
```

```cpp
// Delete node by value
template <typename T>
void dllDeleteNode(T data, DoubleList<T>*& head) {
    if (head == nullptr) return;

    DoubleList<T>* temp = head;

    // Search for the node in circular list
    while (temp->data != data) {
        temp = temp->next;
        if (temp == head) {
            std::cout << "Song \"" << data << "\" not found!" << std::endl;
            return;
        }
    }

    // Only one node in the list
    if (temp->next == temp) {
        head = nullptr;
        delete temp;
        return;
    }

    // Unlink the node
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;

    // If deleting head, move head pointer to next node
    if (temp == head) {
        head = temp->next;
    }

    delete temp;
}
```

```cpp
#include <iostream>
#include <string>
#include "circular_doubly_ll.h"

// Main Program
int main() {

    // Creating initial playlist nodes
    DoubleList<std::string>* head = CreateNewNode(std::string("Twice - Yes or Yes"));
    DoubleList<std::string>* second = CreateNewNode(std::string("Twice - What Is Love?"));
    DoubleList<std::string>* third = CreateNewNode(std::string("Twice - Look At Me Now"));
    DoubleList<std::string>* fourth = CreateNewNode(std::string("Twice - The Feels"));

    // Manual circular linking: fourth->next goes back to head, head->prev points to fourth
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = head; // Circular link

    head->prev = fourth; // Circular link
    second->prev = head;
    third->prev = second;
    fourth->prev = third;

    std::cout << "Try playlist traversal:\n";
    dllListTraversal(head);
    std::cout << std::endl;

    std::cout << "Try inserting song at head:\n";
    dllInsertHead(std::string("Twice - Signal"), head);
    dllListTraversal(head);
    std::cout << std::endl;

    std::cout << "Try inserting song at tail:\n";
    dllInsertTail(std::string("Twice - Heart Shaker"), head);
    dllListTraversal(head);
    std::cout << std::endl;

    std::cout << "Try general insert after 3rd node:\n";
    dllGeneralInsert(std::string("Twice - Likey"), head->next->next);
    dllListTraversal(head);
    std::cout << std::endl;

    std::cout << "Try deleting 'Twice - Likey':\n";
    dllDeleteNode(std::string("Twice - Likey"), head);
    dllListTraversal(head);
    std::cout << std::endl;


    return 0;
}
```

So for this we learned double linked list compared to single linked list it has prev so the head->prev is pointing at nullptr and the tail->next is pointing at nullptr it most of the functions are the same and how you program it it just changed on traversing it because you can traverse it backwards since it has prev and it gets challenging for me to the part delete the node since on the single linked list you only look at the next you dont care about the prev but in this it has a prev so its challenging for me it tested my critical thinking skill. For the supplementary its just the same you just need to point the tail at the head and to the do while instead of != nullptr you'll use head since there's no nullptr here since its circular.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CoPilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept explanation analogy for code and visualizing**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I just used it to explain hard concepts and translate or use it on a analogy that i can understand easier**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.